

ResearchHub

AI-Powered Student Research Assistant

Complete Project Development Document

Final Version · March 2026 · Local Development

Project	ResearchHub
Version	Final — Local Development Edition
Users	Students & Researchers
Paper APIs	arXiv, Semantic Scholar, CrossRef (all free)
Frontend	React 18 + Vite + TailwindCSS + Shadcn/ui
Backend	FastAPI (Python 3.11)
LLM / AI	Groq API — llama3-8b-8192 (free tier)
Database	SQLite — single file, zero installation
Cache	In-memory Python dict — no Redis
Auth	JWT + bcrypt
Agent	Antigravity AI — reads this PDF as its spec
Docker	NOT required — everything runs natively

Table of Contents

1.	Project Overview & Vision
2.	System Architecture
3.	Complete Tech Stack
4.	Database Schema (SQLite)
5.	External Paper API Integration
6.	Backend Development (FastAPI)
7.	AI Layer — Groq LLM Integration
8.	Frontend Development (React)
9.	Project Workflow & Milestones
10.	File & Folder Structure
11.	Local Environment Setup
12.	API Keys — What You Need & Where They Go
13.	Testing Strategy
14.	Security Considerations
15.	Antigravity Agent Instructions

1. Project Overview & Vision

ResearchHub is a one-stop AI-powered research assistant for students. A user types any project idea and the platform fetches 5-10 real peer-reviewed papers from arXiv, Semantic Scholar, and CrossRef, then uses Groq LLM to generate a topic-specific paper summary, project workflow, and tech stack — tailored to that exact idea. Everything runs on your local machine with no Docker, no cloud, and no external database server.

Core Features

Feature	Description
Paper Discovery	Queries arXiv + Semantic Scholar + CrossRef simultaneously. Returns 5-10 real papers ranked by citation count for any topic the user types.
AI Summaries	Groq LLM reads each paper's title and abstract and writes a clear 3-sentence summary: problem, method, and key result.
AI Workflow	Groq generates a phased development plan specific to the student's exact project idea — not a hardcoded template.
AI Tech Stack	Groq recommends languages, frameworks, libraries, datasets, and tools based on the actual project idea.
Save & History	Users bookmark results, add personal notes, and revisit all past research sessions stored in SQLite.

2. System Architecture

Three processes run on your local machine. The React frontend (Vite) talks to FastAPI over localhost. FastAPI stores all data in a single SQLite file. Groq API calls go out to the internet for AI generation. Paper API calls go out for research data.

Layer	Technology	Where
Browser UI	React 18 + Vite	localhost:5173
API Server	FastAPI + Uvicorn	localhost:8000
Business Logic	Python services	Inside FastAPI
AI / LLM	Groq API (llama3-8b-8192)	Groq cloud — free
Paper APIs	arXiv, Semantic Scholar, CrossRef	External — all free
Database	SQLite via aiosqlite	backend/researchhub.db
Cache	In-memory Python dict + TTL	Inside FastAPI
Auth	JWT + bcrypt	Inside FastAPI

Start the Project (2 terminals only)

```
# Terminal 1 – Backend
cd backend && uvicorn app.main:app --reload

# Terminal 2 – Frontend
cd frontend && npm run dev

# SQLite DB is created automatically. No other setup needed.
```

3. Complete Tech Stack

Frontend

- React 18 — UI framework
- Vite — local dev server with hot reload
- TailwindCSS 3 — utility-first styling
- Shadcn/ui — accessible component library
- React Router v6 — client-side routing
- Zustand — global state (user session, token)
- Axios — HTTP client with JWT interceptor
- React Query (TanStack) — server state, loading, caching
- Framer Motion — animations and page transitions
- Lucide React — icon set

Backend

- Python 3.11+
- FastAPI — async REST framework, auto Swagger docs at /docs
- Uvicorn — ASGI server
- SQLAlchemy 2.0 async — ORM
- aiosqlite — async SQLite driver
- Alembic — database schema migrations
- Pydantic v2 — request/response validation
- httpx — async HTTP for paper APIs
- groq — official Groq Python SDK
- python-jose — JWT tokens
- passlib[bcrypt] — password hashing
- slowapi — rate limiting
- python-dotenv — .env loader

AI — Groq (free tier)

- Model: llama3-8b-8192 — fast, high quality, free
- Understands any project idea — not keyword matching
- Generates: per-paper summaries, project workflow, tech stack
- Free tier: 14,400 requests/day, 30 requests/minute
- Sign up free at console.groq.com — no credit card

Paper APIs — all free

- arXiv API — no key required
- Semantic Scholar API — free tier, optional key for higher limits

- CrossRef REST API — no key required

Database & Cache

- SQLite — zero installation, single file (researchhub.db)
- aiosqlite — makes SQLite compatible with async FastAPI
- In-memory Python dict — replaces Redis, built into FastAPI process

4. Database Schema (SQLite)

Four tables in a single local file. SQLAlchemy + Alembic create everything automatically on first run — no manual SQL needed.

Table: users

```
id INTEGER PRIMARY KEY AUTOINCREMENT
email VARCHAR(255) UNIQUE NOT NULL
hashed_password VARCHAR NOT NULL
full_name VARCHAR(100)
created_at DATETIME DEFAULT (datetime('now'))
is_active BOOLEAN DEFAULT 1
```

Table: research_queries

```
id INTEGER PRIMARY KEY AUTOINCREMENT
user_id INTEGER REFERENCES users(id) ON DELETE CASCADE
query_text TEXT NOT NULL
created_at DATETIME DEFAULT (datetime('now'))
workflow TEXT -- JSON string
tech_stack TEXT -- JSON string
```

Table: research_papers

```
id INTEGER PRIMARY KEY AUTOINCREMENT
query_id INTEGER REFERENCES research_queries(id) ON DELETE CASCADE
title TEXT NOT NULL
authors TEXT -- JSON array string
abstract TEXT
ai_summary TEXT -- Groq generated
arxiv_id VARCHAR(30)
doi VARCHAR(120)
published_date VARCHAR(20)
citation_count INTEGER DEFAULT 0
pdf_url TEXT
source VARCHAR(30) -- arxiv | semantic_scholar | crossref
```

Table: saved_results

```
id INTEGER PRIMARY KEY AUTOINCREMENT
user_id INTEGER REFERENCES users(id)
query_id INTEGER REFERENCES research_queries(id)
saved_at DATETIME DEFAULT (datetime('now'))
notes TEXT
```

Connection string for SQLite in config.py

```
DATABASE_URL = 'sqlite+aiosqlite:///./researchhub.db'
```

5. External Paper API Integration

arXiv API — No Key Required

```
GET http://export.arxiv.org/api/query
```

```
?search_query=all:{QUERY}&max_results=10&sortBy=relevance
```

Returns Atom XML. Parse with `xml.etree.ElementTree`. Extract: `id`, `title`, `summary`, `author`, `published`, `pdf link`.

Semantic Scholar API — Optional Key

```
GET https://api.semanticscholar.org/graph/v1/paper/search
```

```
?query={QUERY}&fields=title,authors,abstract,citationCount,year,openAccessPdf
```

```
Header: x-api-key: {SEMANTIC_SCHOLAR_API_KEY} # optional
```

Without key: 100 requests per 5 min. With key: 10,000 per day.

CrossRef API — No Key Required

```
GET https://api.crossref.org/works?query={QUERY}&rows=10
```

```
Header: User-Agent: ResearchHub/1.0 (mailto:your@email.com)
```

Aggregation Logic

- Call all three APIs simultaneously with `asyncio.gather()` — never sequentially.
- Normalise each result to the standard paper dict (see below).
- Deduplicate: first by DOI, then by `arxiv_id`.
- Sort merged list by `citation_count` descending.
- Return top N (default 10, range 5-10).
- If any one API fails — catch exception, log, continue with the other two.

Normalised Paper Dict — All Services Return This Shape

```
{
  "title": str,
  "authors": list[str],
  "abstract": str,
  "published_date": str, # "YYYY-MM-DD"
  "citation_count": int,
  "pdf_url": str | None,
  "doi": str | None,
  "arxiv_id": str | None,
  "source": str, # "arxiv"|"semantic_scholar"|"crossref"
  "url": str,
}
```

6. Backend Development (FastAPI)

REST API Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	Public	Create user account
POST	/api/auth/login	Public	Returns JWT token
GET	/api/auth/me	JWT	Current user profile
POST	/api/research/search	JWT	Main — fetch papers + Groq AI
GET	/api/research/history	JWT	All past queries for user
GET	/api/research/{id}	JWT	Single query result
DELETE	/api/research/{id}	JWT	Delete a query
POST	/api/saved/	JWT	Save result to dashboard
GET	/api/saved/	JWT	List saved results
DELETE	/api/saved/{id}	JWT	Remove from saved
GET	/api/health	Public	Health check

Search Endpoint Flow — POST /api/research/search

- Step 1: Verify JWT, extract user_id.
- Step 2: Validate body: { query: str (max 500 chars), max_results: int = 10 }.
- Step 3: Check in-memory cache — return immediately if hit (TTL 1 hour).
- Step 4: asyncio.gather() — fetch arXiv + Semantic Scholar + CrossRef in parallel.
- Step 5: Deduplicate by DOI then arxiv_id, sort by citation_count desc, slice to max_results.
- Step 6: await ai_service.generate_all(query, papers) — single Groq call.
- Step 7: Attach ai_summary to each paper dict from Groq response.
- Step 8: Save ResearchQuery + all ResearchPaper rows to SQLite.
- Step 9: Store full response in in-memory cache (TTL: 3600s).
- Step 10: Return: { query_id, query, papers, workflow, tech_stack, created_at }.

7. AI Layer — Groq LLM Integration

ai_service.py makes one Groq API call per search and returns all three outputs — summaries, workflow, and tech stack — as a single JSON object. Groq actually understands the project idea, so every response is genuinely specific to what the student typed.

Install & Setup

```
pip install groq
# Get free key at: https://console.groq.com (no credit card)
# Add to .env: GROQ_API_KEY=gsk_...
```

ai_service.py — Complete Implementation

```
from groq import Groq
import json, os

client = Groq(api_key=os.environ['GROQ_API_KEY'])
MODEL = 'llama3-8b-8192'
SYSTEM_PROMPT = '''
You are an expert research assistant and software architect helping students.
Given a project idea and related papers, return a single valid JSON with three keys:
summaries: list of {paper_index: int, summary: str}
-- summary = 3 sentences: (1) problem, (2) method, (3) result
workflow: {phases: [{phase:int, name:str, description:str,
tasks:[str], estimated_duration:str}]}
-- specific to this exact project idea, not generic
tech_stack: {languages:[str], frameworks:[str], libraries:[str],
datasets:[str], tools:[str]}
-- specific to this exact project idea
Return ONLY the JSON. No explanation, no markdown, no backticks.
'''

async def generate_all(query: str, papers: list[dict]) -> dict:
    paper_info = [
        {'index': i, 'title': p['title'],
        'abstract': (p.get('abstract') or '')[:400]}
        for i, p in enumerate(papers)
    ]
    prompt = f'Project idea: {query}\n\nPapers:\n{json.dumps(paper_info, indent=2)}'
    try:
        resp = client.chat.completions.create(
            model=MODEL,
            messages=[
                {'role': 'system', 'content': SYSTEM_PROMPT},
                {'role': 'user', 'content': prompt},
            ],
            max_tokens=2000, temperature=0.4,
        )
        return _parse(resp.choices[0].message.content)
    except Exception as e:
        print(f'Groq error: {e}')
```

```

return {'summaries': [], 'workflow': {}, 'tech_stack': {}}
def _parse(raw: str) -> dict:
try:
clean = raw.strip()
clean = clean.removeprefix('```json').removeprefix('```')
clean = clean.removesuffix('```').strip()
data = json.loads(clean)
return {'summaries': data.get('summaries', []),
'workflow': data.get('workflow', {}),
'tech_stack': data.get('tech_stack', {})}
except Exception:
return {'summaries': [], 'workflow': {}, 'tech_stack': {}}

```

In-Memory Cache — cache.py

```

import time
from typing import Any
_store: dict[str, tuple[Any, float]] = {}
def get(key: str, ttl: int = 3600) -> Any | None:
if key in _store:
val, ts = _store[key]
if time.time() - ts < ttl:
return val
del _store[key]
return None
def set(key: str, value: Any) -> None:
_store[key] = (value, time.time())

```

8. Frontend Development (React)

Pages & Routes

Route	Page	Description
/	Landing	Hero, feature cards, example query, login/register CTA
/login	Login	Email + password, JWT stored in Zustand on success
/register	Register	Sign-up with validation
/search	Search	Main page — search bar, paper cards, workflow panel, tech stack grid
/history	History	All past queries, click to reload full results
/saved	Saved	Bookmarked searches, personal notes, remove option
/paper/:id	Paper	Full paper detail — metadata, Groq summary, PDF link

Key Components

- — input, loading spinner, submit on Enter or button click
- — title, authors, year, citation badge, Groq summary in accordion
- — numbered phases, icon, description, task checklist per phase
- — badge chips: Languages / Frameworks / Libraries / Datasets / Tools
- — skeleton cards that match real card layout exactly
- — Zustand context: user, token, login(), logout()
- — redirect to /login if no JWT in store
- — success and error toast notifications

Vite Proxy — eliminates CORS issues in dev

```
// vite.config.js
export default defineConfig({
  server: { proxy: { '/api': 'http://localhost:8000' } }
})
```

Axios Client — JWT auto-attached

```
// src/api/client.js
import axios from 'axios'
import { useAuthStore } from '../store/authStore'
const client = axios.create({ baseURL: '/api' })
client.interceptors.request.use(cfg => {
  const token = useAuthStore.getState().token
  if (token) cfg.headers.Authorization = `Bearer ${token}`
  return cfg
})
export default client
```

9. Project Workflow & Milestones

Phase	Name	Days	Key Deliverables
1	Scaffold	1	Folder structure, .env.example, requirements.txt, package.json, SQLite config, FastAPI app factory, Vite scaffold
2	DB & Auth	2-3	SQLAlchemy models, Alembic migration, register/login/me endpoints, JWT, bcrypt, Pytest auth tests
3	Paper APIs	4-6	arxiv_service.py, semantic_scholar_service.py, crossref_service.py, aggregator_service.py, unit tests (mocked httpx)
4	Groq AI	7	ai_service.py — system prompt, Groq call, JSON parser, error handling, unit tests with mocked Groq client
5	Search Endpoint	8	POST /api/research/search — full 10-step flow, in-memory cache, SQLite persistence, integration tests
6	Other Endpoints	9	History, saved CRUD, paper detail, health check. Swagger docs verified at localhost:8000/docs
7	React Scaffold	10-11	Vite + Tailwind + Shadcn/ui + React Router + Zustand + Axios client + React Query + AuthProvider + ProtectedRoute
8	UI Components	12-14	Build SearchBar, PaperCard, WorkflowPanel, TechStackGrid, LoadingState with mock data first — no API calls yet
9	Wire to API	15-16	Connect all components to real backend. Handle loading, error, and empty states on every page
10	Polish & QA	17-19	Framer Motion animations, mobile responsive grid (1→2→3 cols), full Pytest + Vitest suites, manual QA

10. File & Folder Structure

```
researchhub/
├── .env # your secrets (gitignored)
├── .env.example # placeholder template
├── README.md
├──
├── backend/
│   ├── researchhub.db # SQLite – auto-created on first run
│   ├── app/
│   │   ├── main.py # FastAPI app, CORS, routers, lifespan
│   │   ├── config.py # pydantic-settings, reads .env
│   │   ├── database.py # async SQLAlchemy engine (aiosqlite)
│   │   ├── cache.py # in-memory TTL dict cache
│   │   ├── models/
│   │   │   ├── user.py
│   │   │   ├── query.py
│   │   │   ├── paper.py
│   │   │   ├── schemas/
│   │   │   │   ├── auth.py
│   │   │   │   ├── research.py
│   │   │   │   ├── saved.py
│   │   │   ├── routers/
│   │   │   │   ├── auth.py
│   │   │   │   ├── research.py
│   │   │   │   ├── saved.py
│   │   │   ├── services/
│   │   │   │   ├── arxiv_service.py
│   │   │   │   ├── semantic_scholar_service.py
│   │   │   │   ├── crossref_service.py
│   │   │   │   ├── aggregator_service.py
│   │   │   │   ├── ai_service.py # Groq LLM integration
│   │   │   ├── core/
│   │   │   │   ├── security.py # JWT + bcrypt
│   │   │   │   ├── dependencies.py # get_current_user dependency
│   │   │   ├── alembic/
│   │   │   ├── versions/
│   │   │   ├── tests/
│   │   │   │   ├── test_auth.py
│   │   │   │   ├── test_research.py
│   │   │   │   ├── test_services.py
│   │   │   ├── requirements.txt
│   │   │   └──
│   └── frontend/
│       ├── src/
│       │   ├── main.jsx
│       │   ├── App.jsx
│       │   ├── pages/
│       │   │   ├── Landing.jsx
│       │   │   ├── Search.jsx
```

```
■ ■ ■■ History.jsx
■ ■ ■■ Saved.jsx
■ ■■ components/
■ ■ ■■ SearchBar.jsx
■ ■ ■■ PaperCard.jsx
■ ■ ■■ WorkflowPanel.jsx
■ ■ ■■ TechStackGrid.jsx
■ ■ ■■ LoadingState.jsx
■ ■ ■■ Toaster.jsx
■ ■■ store/
■ ■ ■■ authStore.js
■ ■■ api/
■ ■ ■■ client.js # Axios + JWT interceptor
■ ■ ■■ research.js # React Query hook functions
■ ■■ utils/
■■■ index.html
■■■ vite.config.js # /api proxy to localhost:8000
■■■ tailwind.config.js
```

11. Local Environment Setup

Prerequisites — Install Once on Your Machine

- Python 3.11+ — python.org
- Node.js 18+ — nodejs.org
- Antigravity — your AI coding agent
- Groq free API key — console.groq.com (2 minutes, no credit card)

requirements.txt — Backend

```
fastapi
uvicorn[standard]
sqlalchemy[asyncio]
aiosqlite
alembic
pydantic-settings
httpx
python-jose[cryptography]
passlib[bcrypt]
slowapi
python-dotenv
groq
pytest
pytest-asyncio
```

First-Time Setup Commands

```
git clone https://github.com/yourname/researchhub && cd researchhub
cp .env.example .env # fill in your keys
# Backend
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
alembic upgrade head # creates researchhub.db
uvicorn app.main:app --reload
# Frontend (new terminal)
cd frontend
npm install
npm run dev
# Verify:
# http://localhost:5173 -> React app
# http://localhost:8000/docs -> Swagger API docs
```

12. API Keys — What You Need & Where They Go

Once the project is fully built and running, the agent must display this checklist to the developer so they know exactly which keys to get and where to put them.

Complete API Key Reference

Key Name	Required?	Where to Get It	Where to Put It
GROQ_API_KEY	YES	console.groq.com — free, no credit card	.env → GROQ_API_KEY=gsk_...
SECRET_KEY	YES	Generate any random 32+ char string	.env → SECRET_KEY=...
SEMANTIC_SCHOLAR_API_KEY	Optional	semanticscholar.org/product/api	.env → SEMANTIC_SCHOLAR_API_KEY=...

arXiv and CrossRef require NO API key at all.

Complete .env File

```
# ■■ REQUIRED
GROQ_API_KEY=gsk_your_key_from_console_groq_com
SECRET_KEY=make-this-any-long-random-string-32chars-min
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# ■■ DATABASE (no changes needed for SQLite)
DATABASE_URL=sqlite+aiosqlite:///./researchhub.db

# ■■ OPTIONAL (improves Semantic Scholar rate limits)
SEMANTIC_SCHOLAR_API_KEY=your_key_here
ENVIRONMENT=development
```

Agent: After Build is Complete — Show This Message

When the project is fully working, print the following checklist to the terminal so the developer knows exactly what to configure:

```

=== ResearchHub is ready! Configure your API keys: ===

1. GROQ_API_KEY (REQUIRED)
Get it free at: https://console.groq.com
Add to: backend/.env as GROQ_API_KEY=gsk...

2. SECRET_KEY (REQUIRED)
Generate one: python -c "import secrets; print(secrets.token_hex(32))"
Add to: backend/.env as SECRET_KEY=...

3. SEMANTIC_SCHOLAR_API_KEY (optional but recommended)
Get it at: https://www.semanticscholar.org/product/api
Add to: backend/.env as SEMANTIC_SCHOLAR_API_KEY=...

arXiv and CrossRef need NO keys – they work immediately.

After adding keys, restart the backend:

uvicorn app.main:app --reload

```

13. Testing Strategy

Backend — Pytest

- Unit tests per paper API service — mock httpx responses with pytest-httpx.
- Unit tests for ai_service.py — mock Groq client, test prompt building and JSON parsing.
- Integration tests for all FastAPI endpoints using FastAPI TestClient.
- Tests use in-memory SQLite (:memory:) — never touches researchhub.db.

```
cd backend && pytest tests/ -v --cov=app --cov-report=term-missing
```

Frontend — Vitest + React Testing Library

- Component tests for PaperCard, SearchBar, WorkflowPanel with mock props.
- API tests using msw (Mock Service Worker) to intercept fetch calls.

```
cd frontend && npm run test
```

14. Security Considerations

- JWT stored in Zustand in-memory state only — never localStorage (XSS risk).
- CORS set to allow http://localhost:5173 only.
- Rate limit /api/research/search: 10 requests/minute per user (slowapi).
- Query string capped at 500 characters before processing.
- GROQ_API_KEY and SECRET_KEY live only in .env — never sent to frontend.
- Passwords hashed with bcrypt cost factor 12.
- SQLAlchemy parameterised queries prevent SQL injection by default.

15. Antigravity Agent Instructions

This section is written directly for the Antigravity AI agent. Read this entire document before writing any code. This document is your single source of truth.

Implementation Order

- Step 1: Read this entire PDF document completely before writing any code.
- Step 2: Create the full folder structure from Section 10.
- Step 3: Create .env.example with all variables from Section 11.
- Step 4: Create backend/requirements.txt exactly as listed in Section 11.
- Step 5: Write app/config.py (pydantic Settings), app/database.py (aiosqlite engine), app/cache.py (in-memory TTL dict), app/main.py (FastAPI factory with CORS and routers).
- Step 6: Write SQLAlchemy models from Section 4. Run: alembic init, configure env.py for SQLite, alembic revision --autogenerate -m 'init', alembic upgrade head.
- Step 7: Write core/security.py (JWT create/verify + bcrypt hash/verify) and core/dependencies.py (get_current_user FastAPI dependency).
- Step 8: Write routers/auth.py — register, login, /me. Write tests/test_auth.py. Run pytest — all must pass before next phase.
- Step 9: Write services/arxiv_service.py, semantic_scholar_service.py, crossref_service.py. Write services/aggregator_service.py. Write tests/test_services.py with mocked httpx. All tests must pass.
- Step 10: Write services/ai_service.py exactly as shown in Section 7 — Groq client, system prompt, generate_all(), _parse(). Write unit tests mocking the Groq client.
- Step 11: Write routers/research.py — full 10-step search flow from Section 6. Write tests/test_research.py. Run pytest — all pass.
- Step 12: Write routers/saved.py — CRUD for saved results.
- Step 13: Run full backend test suite: pytest tests/ -v. Fix all failures before touching frontend.
- Step 14: Scaffold React frontend: npm create vite@latest frontend -- --template react. Install all packages from Section 3.
- Step 15: Configure vite.config.js proxy and tailwind.config.js. Write Zustand authStore and Axios client with JWT interceptor.
- Step 16: Set up App.jsx with React Router, AuthProvider, ProtectedRoute, and all routes.
- Step 17: Build all components with mock data first: SearchBar, PaperCard, WorkflowPanel, TechStackGrid, LoadingState.
- Step 18: Build all pages: Landing, Search, History, Saved.
- Step 19: Wire all components and pages to real backend API. Handle loading, error, and empty states on every single page.
- Step 20: Add Framer Motion page transitions and staggered card animations. Verify mobile responsive layout.
- Step 21: Run npm run test. Fix all failures.
- Step 22: Do a full manual test: register a new user, search 'Smart Traffic Management using Deep Learning', verify papers appear with Groq summaries, workflow phases, and tech stack badges.

- Step 23: Once everything is confirmed working, print the API Keys checklist from Section 12 to the terminal so the developer knows what to configure.

Non-Negotiable Rules

- NEVER hardcode GROQ_API_KEY or SECRET_KEY anywhere in code. Always use os.environ via config.py.
- NEVER use synchronous requests or httpx in FastAPI routes. Always async httpx.AsyncClient.
- NEVER let one failed paper API crash the whole request. Wrap each service in try/except.
- NEVER store JWT in localStorage. Zustand in-memory only.
- ALWAYS validate Groq JSON output in _parse() before saving to DB. Never crash on bad JSON.
- ALWAYS show skeleton loading screens in React. Never a blank white page while fetching.
- ALWAYS type-annotate every Python function — inputs and return type.
- NO Docker, NO docker-compose.yml, NO Dockerfiles of any kind.
- NO Redis — use cache.py in-memory dict only.
- NO PostgreSQL — SQLite only via sqlite+aiosqlite.
- Run tests after every phase. Do not move to next phase if tests fail.

Done Criteria — All Must Be True

- ✓ uvicorn app.main:app --reload starts with zero errors.
- ✓ npm run dev starts with zero errors.
- ✓ pytest tests/ -v — every single test passes.
- ✓ npm run test — every single test passes.
- ✓ backend/researchhub.db exists after first run (auto-created by Alembic).
- ✓ Typing 'Blockchain-based Voting System' returns real papers + Groq workflow with Smart Contract/Ethereum phases + tech stack with Solidity/Web3.js.
- ✓ Typing 'Image Segmentation using U-Net' returns real papers + Groq workflow with data/preprocessing/training phases + tech stack with PyTorch/OpenCV.
- ✓ API Keys checklist has been printed to the terminal as described in Section 12.

ResearchHub — Final Version — Groq + SQLite + Local Dev + Antigravity

Start at Section 15, Step 1. Build phase by phase. Test before moving forward.